Data & Knowledge Engineering 164 (2026) 102577



Contents lists available at ScienceDirect

Data & Knowledge Engineering

journal homepage: www.elsevier.com/locate/datak





When temporary results meet intermediate index: An optimization technique of procedural SQL query processing

Md Arif Rahman [a,b], Syed Jalaluddin Hashmi [a], Kethsiya Gnanajothy [a], Young-Koo Lee [a,*]

[a] Department of Computer Science and Engineering, Kyung Hee University, 1732, Deogyeong-daero, Giheung-gu, Yongin-si, 17104, Gyeonggi-do, Republic of Korea
[b] Department of Computer Science and Engineering, Jashore University of Science and Technology, Jashore, 7408, Bangladesh

ARTICLE INFO

Keywords:
Intermediate index
Procedural SQL query processing
What-if analysis
Imperative program optimization
AutoCox

ABSTRACT

In modern data-intensive applications, optimizing procedural SQL queries in imperative programs is challenging because of inefficient access to intermediate results that are stored in temporary memory. Both the traditional and machine learning-based optimization techniques often fail to create indexes on intermediate results because they focus on previously stored base tables and declarative queries. However, indexing on intermediate results can be a potential solution to further reducing the cost of processing dependent queries. To the best of our knowledge, no existing work has considered reducing the processing cost of dependent queries that rely on intermediate results. Inspired by this issue, we introduce the intermediate index, a temporary index created on intermediate results within the scope of a single program execution. Leveraging the intermediate index, we propose a procedural SQL query optimization technique called *AutoCox* that identifies and evaluates the benefits of indexes using a novel *what-if* analysis method. *AutoCox* dynamically determines the need for indexing based on the producer–consumer relationships, cardinality, and selectivity of intermediate results, accounting for both index creation overhead and runtime reuse. *AutoCox* ensures that intermediate indexes hold up-to-date data, which are created only when beneficial and automatically dropped afterward. Experimental results show that our approach significantly outperforms existing methods, achieving a magnitude of 67% cost reduction of dependent queries and 261× speedups of the imperative program using a workload. It indicates that we effectively bridge a critical gap in the optimization of procedural SQL queries in relational big data processing environments.

1. Introduction

Although recent advances in machine learning (ML)-based query optimization promise better execution plans, these techniques still rely on traditional cost-based optimizers to incorporate models trained on historical workload patterns [1–5]. The query optimizers often deal with procedural SQL queries, which refer to declarative SQL queries embedded with procedural logic within imperative programs, such as stored procedures and user-defined functions. These programs are widely used to express complex data transformations, encapsulate business rules, and support scalable analytics pipelines. However, in big data environments, optimizing imperative programs remains challenging, especially when intermediate results are materialized in temporary tables

* Corresponding author.
E-mail address: yklee@khu.ac.kr (Y.-K. Lee).

https://doi.org/10.1016/j.datak.2026.102577
Received 2 July 2025; Received in revised form 25 November 2025; Accepted 7 February 2026
Available online 10 February 2026
0169-023X/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

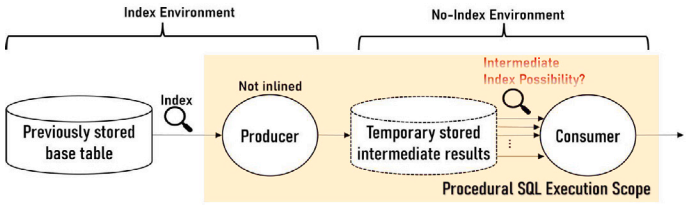


Fig. 1. Intermediate index problem.

or table variables that are accessed repeatedly by procedural SQL queries. Without effective reuse strategies, these intermediate structures often lead to significant performance bottlenecks.

To handle intermediate results efficiently, the producer–consumer relationship using statements plays a pivotal role in optimization. Statements are the basic unit to process intermediate data sources where producer statements generate intermediate results, which are then consumed by consumer statements [6]. We illustrate our problem in Fig. 1. The figure shows a producer statement that filters or aggregates data from a base table and stores the result in a temporary table, which is then accessed by one or more consumer statements. Optimizing this relationship is a trade-off between inlining and materialization.

To optimize access to intermediate results, the optimizer often relies on materialized intermediate results to break down complex operations into manageable steps. Materialization involves storing intermediate results in temporary tables or table variables, which are then accessed by multiple consumer statements. This approach is often preferred over inlining (merging producer and consumer queries) to avoid redundant computation in several scenarios. For example, multiple consumer accesses that define multiple consumer statements or statements within a loop region accessing the same intermediate result. However, finding optimal execution plans by optimizing access to intermediate results presents unique challenges when it comes to accessing them using indexes. Existing auto-indexing methods, such as AutoAdmin [7], HypoPG [8], and Oracle Auto Indexing [9], focus on declarative queries. They do not identify indexes within imperative programs. This limitation leads to the following research objectives.

Research Question 1: Can the optimizer optimize access to intermediate results using index? Optimizing access to intermediate results efficiently using indexes increases the performance of consumer statements. However, without appropriate indexing, performance degrades due to the index not being able to facilitate consumer statements.

Research Question 2: Can the optimizer estimate the benefit of an index created on intermediate results? Indexing intermediate results is not straightforward due to their temporary and dynamic nature. Intermediate results are produced at runtime and vary depending on the current state of the database and input parameters of the program. This means that the indexes for the intermediate results cannot be constructed or evaluated before the program is executed. Hence, to provide index benefits for consumer statements, we must evaluate the potential benefits of creating indexes in a no-index environment. This requires a new approach to *what-if* analysis that considers the temporary nature of intermediate results and the overhead of index creation within the scope of the imperative program.

Current techniques for evaluating index benefits, such as *what-if* analysis in hypothetical indexing methods [10], are designed for base tables that are stored in the database and evaluate the benefits of indexes for stand-alone queries in a workload. They do not address the unique challenges of intermediate results in imperative programs. Moreover, current approaches create indexes outside the scope of query execution, ignoring the overhead of index creation within the program. This is a critical limitation, as the cost of creating indexes on temporary tables must be considered in the benefit analysis.

To the best of our knowledge, we found no prior works that evaluated the benefit analysis of index creation based on intermediate results within the scope of an imperative program. Inspired by the issue, we introduce a concept of an *intermediate index*, which is created on the intermediate results of producer statements to facilitate their consumer statements. The index leverages filtering and joining conditions present in consumer statements to enhance execution efficiency. We propose a *what-if* analysis method, called *AutoCox*, for evaluating the benefits of intermediate indexes within the scope of imperative programs. We employ a cost model in the *what-if* analysis method to estimate the benefits of creating an intermediate index. This model considers the index creation cost that estimates the overhead cost of creating the index on the temporary table and the consumer statement cost that estimates the cost of consumer statements with and without the index, using cardinality estimation and selectivity estimation for filter and join conditions.

The benefits of the intermediate index are: (i) it dynamically focuses directly on consumer statements to optimize access to intermediate results, (ii) it is created immediately after the producer statements, ensuring intermediate results are not further modified by other consumer statements, and (iii) it is automatically created and dropped within the scope of the program, ensuring efficient access without long-term maintenance overhead. Moreover, if the decision to create indexes on intermediate results is left to the model training phase, which relies on a static index creation method rather than our approach, it leads to uncertain benefits because intermediate results are highly sensitive to input parameters. For example, if the cardinality of an intermediate result is large for a parameter setting, creating an index significantly improves performance. On the contrary, if the cardinality is small, the overhead of creating and maintaining the index outweighs the benefits. Our approach addresses this uncertainty by providing a systematic decision for evaluating the benefits of intermediate indexes, ensuring that indexes are only created when they provide a net performance improvement.

The core contributions of this study can be followed as:

- Introducing the *intermediate index*, a dynamic index created on the intermediate results of the producer statement to facilitate consumer statements within the scope of the procedural SQL queries.
- Proposing *AutoCox*, an optimization technique based on *what-if* analysis, in which we develop a cost model to estimate the benefits of creating intermediate indexes for achieving a better execution plan while training the ML-based query optimizer.
- Demonstrating the effectiveness of our proposed approach through extensive experiments. We show that our proposed method improves the performance of imperative programs by up to 261 times compared to state-of-the-art techniques using a workload.

We organize our remaining sections as follows: In Section 2, we discuss the necessary terminologies for the study. In Section 3, we discuss our proposed optimization. Section 4 demonstrates our experiments. In Section 5, we discuss the related studies. Finally, our discussion concludes in Section 6.

2. Preliminaries

To understand our proposed method, we need to know the best way to represent the execution plan of the procedural SQL query where intermediate results are properly tracked. Since our proposed method works on materialization of temporary results where producing queries are not inlined, we analyze which of them are inlined and materialized. Subsequently, we present a formal definition of the intermediate index along with the necessary terminologies that are essential for understanding our proposed method.

2.1. Execution plan representation for intermediate results analysis

We model the execution plan of an imperative program using a statement graph [6]. Statements are the fundamental unit for handling intermediate results in this model. A statement graph is basically a control flow graph (CFG) of the program [11] that captures all possible execution paths through its statements. We define each statement as an assignment statement, represented as a 2-tuple (v_s, e_s), in which e_s is an expression and v_s is a variable assigned the outcome of e_s. An SQL assignment statement results when e_s is a SELECT query expression since the resultant relation is assigned to v_s. Under this situation, v_s is considered a table variable. We represent and track intermediate results assignment statements. We establish a producer–consumer relationship between statements: a producer generates intermediate results that are subsequently consumed by one or more consumer statements. We also reflect common table expressions (CTEs) and subqueries using assignment statements that allow a uniform approach to all intermediate computations. We decompose loop-invariant statements and hoist them outside the loop to be performed once to maximize loop processing; loop-variant statements stay inside the loop for recurrent use.

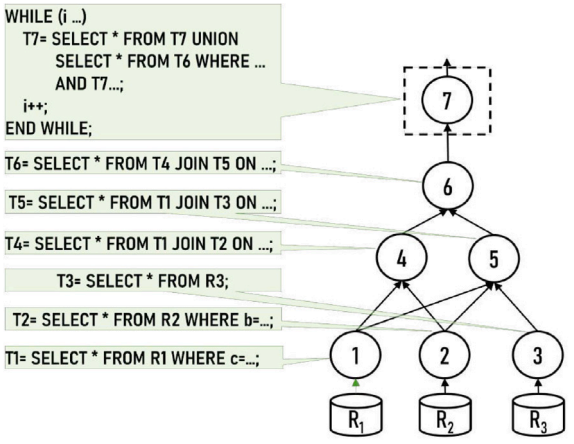
We categorize statements into two distinct categories: (i) Declarative statements including SELECT, INSERT, UPDATE, and DELETE that act on intermediate results and (ii) imperative statements including the headers and bodies of loops and branches, which define control flow logic, guide the traversal and processing of statements inside the graph. In Fig. 2, we represent a statement graph of a procedural SQL query. In Fig. 2(a), we present an example program that contains three base table scans, two joins, and a loop region. We represent the program using a statement graph in Fig. 2(b). We decompose each intermediate data access using declarative SQL statements, such as statements 1 to 7. We combine imperative statements with declarative statements, such as statement 7. In the graph, circles represent declarative statements. The directed lines represent data and control flow dependencies where data flows from producers to consumers. Early on in the imperative program optimization process, inlining became a common method to combine consumer and producer queries, hence lowering overhead. Eliminating intermediate result storage helps to increase speed for small datasets and reduces the requirement for temporary tables. However inlining lost effectiveness as programs become more complicated, particularly when intermediate results were substantial or repeated over several consumer searches. This led to the acceptance of materialization, in which temporary tables or table variables hold intermediate results. Materialization makes it possible to step-by-step handle challenging tasks.

2.2. Inlining analysis

Inlining is a technique of directly substituting a producer statement with corresponding consumer statements. Instead of executing multiple dependent statements independently, it essentially merges them into a single statement. This change allows on-the-fly computation of intermediate results, therefore preventing their materialization. Let formally $s_1 = (v_1, e_1)$ and $s_2 = (v_2, e_2)$ be two different SQL assignment statements inside the statement graph. SQL statement inlining substitutes the expression e_1, i.e., $v_1 \in \text{refs}(e_2) \Rightarrow e_2' := e_2[v_1 \mapsto e_1]$. This substitution is valid if and only if there exists a data-flow dependency from s_1 to s_2, denoted $s_1 \xrightarrow{\text{Data Flows}} s_2$. In this instance, s_2 is said to be an inlined statement. Inlining is particularly beneficial for SELECT queries, as it exposes more structure within the query expression, enabling the query planner to apply algebraic optimizations, including join reordering for improved join plan selection, Predicate pushdown to reduce data volume earlier in the pipeline and group-by simplification to streamline aggregations. In Fig. 2(b), for example, statement 2 is the producer statement of statement 4. If inlining is beneficial, the optimizer merges statements 2 and 4 in statement 4.

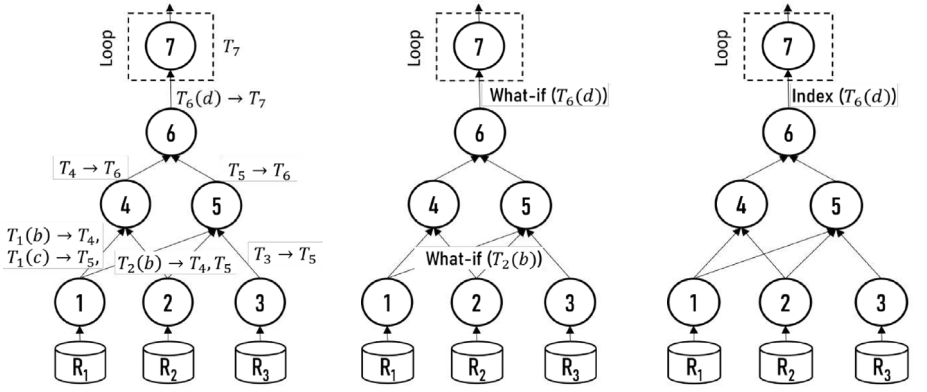
```
CREATE PROCEDURE example_1 (...) AS BEGIN ...
 DECLARE T7 ...
 i=...;
 FOR (i ...)
   T7 = SELECT * FROM T7 UNION SELECT * FROM T1
     JOIN T2 ON ... JOIN T3 ON ...
     WHERE T1.c =... AND T2.b =...AND T7...;
    i = ...;
 END FOR;
END;
```

(a) Procedural SQL program.



(b) Statement graph.

Fig. 2. Execution plan representation.



(a) Statement graph.

(b) Potential indexes.

(c) *What-if* Analysis to find beneficial index.

Fig. 3. Intermediate index optimization process.

2.3. Materialization analysis

Materialization refers to the process of storing intermediate results that are produced by a statement. The goal is to reuse them across multiple consumer statements, especially when recomputing them is expensive. This approach avoids inlining. Materialization incurs additional memory/storage overhead. However, it reduces the total execution cost when the intermediate result is accessed repeatedly. We apply materialization selectively, only when it is cost-effective compared to inlining. Let $s_1 = (v_1, e_1)$ be a producer statement, and let s_2 be a consumer statement that references v_1 n times. Let m_1 be the cost of materializing the results of s_1, cost(s_1) be the cost of evaluating e_1, cost(s_2) be the cost of evaluating s_2 using the materialized result, cost(inlined s_2) be the cost of evaluating s_2 after inlining e_1 in place of v_1. We prefer materialization when the total cost of recomputing via inlining exceeds the cost of evaluating and storing the result once and reusing it, i.e., $\sum_{i=1}^{n} \text{cost}(\text{inlined } s_2^{(i)}) > m_1 + \text{cost}(s_1) + \sum_{i=1}^{n} \text{cost}(s_2^{(i)})$. In this formulation, materialization reduces redundant computation at the cost of memory usage, and it is triggered only when the reuse benefit outweighs this cost. In Fig. 2(b), for example, statement 6 is accessed by statement 7 multiple times. If computation of statement 6 is very expensive, we do not inline statement 6 into its consumer statement 7; rather, we materialize statement 6. Materialized intermediate results are ephemeral; they expire at the end of the program execution, ensuring that each program run processes up-to-date data. This guarantees correctness and avoids unintended reuse across executions.

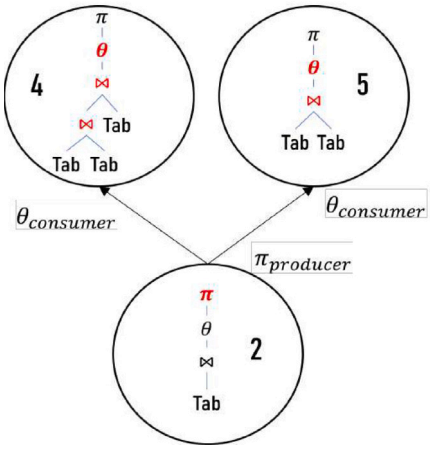


Fig. 4. *Autocox* intermediate index identification.

2.4. Formal definition of intermediate index

We formally define the intermediate index ($IIndex$) as:

$IIndex(\pi_{producer}, \theta_{consumer})$ where $\theta_{consumer} \in \pi_{producer}$

Here $\pi_{producer}$ represents the set of columns projected by the producer statement. $\theta_{consumer}$ represents the set of columns used in filtering and joining conditions in consumer statements. The condition $\theta_{consumer} \in \pi_{producer}$ ensures that the index is created only on columns that exist in the producer's output.

3. Proposed optimization

Our optimization process begins after the optimizer determines which temporary tables are inlined or materialized from the statement graph. We apply *AutoCox* to identify an intermediate index on a materialized temporary table. It then performs a *what-if* analysis to evaluate the benefits of creating intermediate indexes on materialized temporary tables. If a beneficial index is identified, it is introduced after the corresponding statement in the physical execution plan. Fig. 3 illustrates our optimization method. Fig. 3(a) presents the statement graph that is the same as Fig. 2(b). We assume the optimizer selects temporary tables T_2 and T_6 (from Statements 2 and 6) for materialization due to multiple consumer accesses. This assumption reflects the standard behavior of both traditional and learned optimizers, which materialize temporary results when: (i) the producer statement has high evaluation cost, and (ii) the result is accessed by multiple consumer statements. In our example, T_2 and T_6 satisfy these criteria; thus materialization represents the optimal choice under the cost model. We then apply a *what-if* analysis to assess intermediate indexes on T_2 and T_6, as shown in Fig. 3(b). If a beneficial index is found on T_6, we create it after Statement 6 in the execution plan. The corresponding physical plan is depicted in Fig. 3(c). Notably, the optimizer may choose inlining even for temporary tables with multiple consumer accesses. For instance, Table T_1 is not materialized despite having multiple consumers.

3.1. AutoCox: Consumer-aware auto-indexing

AutoCox is a novel approach that creates hypothetical intermediate indexes on the intermediate results of producer statements. It identifies filtering and joining conditions in consumer statements that correspond to columns projected by the producer statement. Fig. 4 illustrates *AutoCox*'s intermediate index identification. In this example, we create an intermediate index on the temporary table of statement 2. It benefits its consumer statements 4 and 5. *AutoCox* leverages the projected columns of statement 2 in the selection and join conditions from statements 4 and 5 to optimize query execution. *AutoCox* has three phases to identify a hypothetical intermediate index: (i) program analysis, (ii) index identification and generation phase, and (iii) index recommendation phase. We formally define the methodology of *AutoCox* as follows:

In the program analysis phase, *AutoCox* analyzes the statement graph and extracts the following information: (i) Producer Statements (*producer*): Identifies temporary tables generated during execution. (ii) Consumer statements (*consumer*): identifies the fragments of statements that reference producer statements. (iii) Projection columns ($\pi_{producer}$): Determines columns that are output by each producer. (iv) Filtering and Joining Conditions ($\theta_{consumer}$): Identifies predicates (e.g., WHERE, JOIN ON) in consumer statements.

In the index identification and generation phase, *AutoCox* performs the following steps: Step 1: Condition Matching. It checks whether the filtering or joining conditions ($\theta_{consumer}$) reference columns from the producer's projection ($\pi_{producer}$) match each

Algorithm 1 Algorithm of *AutoCox*.

Input: Logical Plan ($Plan$)
Output: Intermediate Indexes ($IIndex$)

```
AutoCox(Plan):
    // Step 1: Extract producer and consumer statements
    producers = extractProducerStatements(Plan)
    consumers = extractConsumerStatements(Plan)
    // Step 2: Identify candidate columns in temporary tables for indexing, where temporary tables materialized due to multiple consumer accesses.
    FOREACH producer in producers:
        tempTable = producer.outputTable
        projectedColumns = producer.projection
        FOREACH consumer in consumers:
            if consumer.references(tempTable) then
                filterConditions = consumer.filterConditions
                joinConditions = consumer.joinConditions
                indexColumns = filterConditions ∪ joinConditions

                // Step 3: Apply indexing rules
                if indexColumns ⊆ projectedColumns then
                    if multipleConditions(indexColumns) then
                        createCompositeIndex(tempTable, indexColumns)
                    else
                        if sharedConditionExists(indexColumns, consumers) then
                            createSharedIndex(tempTable, indexColumns)
                        else
                            createIndex(tempTable, indexColumns)
                        end
                    end
                else
                    continue // No valid index conditions found
                end
            end
    //Step 4: Output hypothetical indexes
    return listIntermediateIndexes()
```

consumer statement ($consumer_i$). These columns are eligible for indexing where $\theta_{consumer} \subseteq \pi_{producer}$. Step 2: Applied Rules: (i) *Composite Index Creation*: If there are multiple conditions in a single consumer statement, we create a composite index on those columns. (ii) *Shared Index for Reused Conditions*: If the same condition is in multiple consumer statements, we create one index instead of multiple. (iii) *Indexing Multiple Materialized Tables*: If there are multiple consumer statements that reference multiple producer statements, we create a separate index per producer. (iv) *No Index Creation*: If a consumer statement has no filtering or joining condition, we skip indexing. Step 3: Hypothetical Index Generation. It defines $IIndex(\pi_{producer}, \theta_{consumer})$. In the index recommendation phase, *AutoCox* outputs a set of hypothetical indexes that improve query performance. We use these indexes in a *what-if* analysis to determine their impact on execution plans.

We show the process in Algorithm 1. We first identify and create hypothetical indexes for consumer statements to access producer results. *extractProducerStatements(Plan)* is responsible for retrieving producer statements that generate temporary tables. *extractConsumerStatements(Plan)* is responsible for identifying consumer statements that access producer results. *consumer.references(tempTable)* is responsible for determining a consumer statement that accesses a given temporary table. *multipleConditions(indexColumns)* is responsible for checking whether a consumer statement has multiple filtering/joining conditions for a producer result. *sharedConditionExists(indexColumns, consumers)* is responsible for detecting the same filtering/join condition is used across multiple consumer statements. *createCompositeIndex(tempTable, indexColumns)* is responsible for creating a composite index when multiple filtering/join conditions exist. *createSharedIndex(tempTable, indexColumns)* is responsible for creating a shared index when multiple consumers use the same filtering/join condition. *createIndex(tempTable, indexColumns)* is responsible for creating an index for a single filtering/join condition. *listIntermediateIndexes()* returns all created hypothetical indexes for evaluation.

Consider the temporary table produced by Statement 6 in Fig. 3. Statements 4 and 5 consume this table using predicates on column d. *AutoCox* identifies (d) as the candidate indexing column, generates a hypothetical index $IIndex(T_6, d)$, and triggers *what-if* analysis. Algorithm 2 then evaluates the full-scan cost of each consumer, computes the index-creation overhead, and finally compares the cumulative benefit. Because the consumer statements are repeatedly executed in a loop, the index is deemed beneficial and inserted after Statement 6 in the final physical plan.

Integration With Existing Optimizers: *AutoCox* operates at the logical-plan level and is therefore compatible with both commercial and open-source database systems. The framework can be integrated as a pre-optimization pass that annotates the plan with hypothetical intermediate indexes before the physical optimizer performs cost-based plan selection. Since *AutoCox* does not require

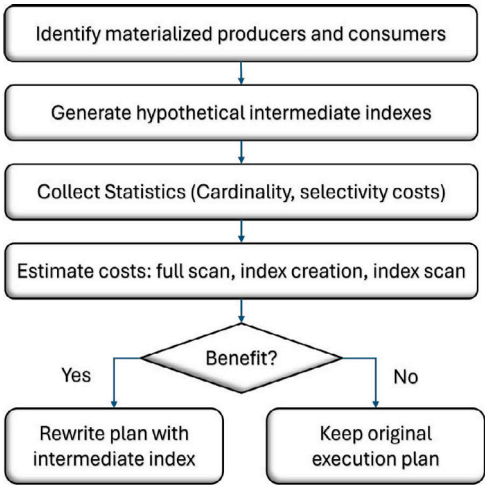


Fig. 5. *What-if* analysis workflow.

modifying the storage engine or physical operators, it can be plugged into PostgreSQL, SQL Server, or Oracle optimizers, as well as learned optimizers such as Neo, Bao, or Balsa. This design ensures broad applicability across existing relational database systems.

3.2. What-if analysis and cost model

What-if analysis evaluates the cost of an intermediate index and its impact on consumer statements by comparing execution costs with and without the index. Without an index, consumer statements perform a full table scan, whereas with an index, they use an index scan. We identify beneficial indexes for materialized temporary tables where the combined cost of index creation and indexed consumer execution is lower than the cost of execution without an index. This evaluation is guided by a cost model. The benefit of creating an intermediate index is determined by comparing the total execution cost of affected consumer statements with and without the index. An index is considered beneficial if the combined cost of index creation and indexed execution is lower than execution without the index. Before presenting the cost equations, we summarize the complete workflow of our *what-if* analysis in Fig. 5. We first identify materialized producer statements and their consumer statements. We then extract cardinality and selectivity estimates of these statements from the optimizer. Next, we generate hypothetical intermediate indexes using *AutoCox*. After that, we estimate the full-scan cost (without index). We also estimate index-creation cost and index-scan cost. Lastly, we compare the total cost and decide whether the index is beneficial. This workflow contextualizes the equations that follow.

Let S be the set of statements in a program, and let t_s be a temporary table in the statement $s \in S$ that can benefit from an index. Let us $S_c \subset S$ denote the consumer statements affected by the index. We define I_{t_s}: Cost of creating an index on t_s, C_s: cost of consumer statements with a full table scan and C'_s: Cost of consumer statements with an index scan. Our cost model is formulated as:

$$\sum_{s}^{S_c} C_s - (I_{t_s} + \sum_{s}^{S_c} C'_s) > 0 \tag{1}$$

Since *what-if* analysis is performed in a no-index environment, the optimizer lacks statistics on the index. Thus, it cannot estimate I_{t_s} and C'_s. We estimate these costs using cardinality and selectivity estimations from producer and consumer statements. Similarly, we estimate C_s, the cost of full table scans using the same metrics to evaluate the benefit of index creation. To estimate the benefit of an intermediate index, we first retrieve information about materialized producer statements and their consumer statements (including cardinality) from the optimizer. If a consumer statement includes a filter or join condition, we proceed with estimating C_s, I_{t_s}, and C'_s.

Parameter Sensitivity and Source of Estimates: All parameters in the cost model, such as I/O cost, CPU cycles per tuple, and selectivity, are obtained directly from optimizer metadata (e.g., PostgreSQL catalog statistics, runtime estimates, and selectivity heuristics). Because *AutoCox* makes decisions based on relative differences between full-scan and index-scan costs, the model is robust to moderate variation in parameter estimates. Our sensitivity analysis confirms that small fluctuations in these values do not affect the correctness of index-benefit decisions.

3.2.1. Estimation of consumer statements cost with a full table scan

Let $size_{c_s}$ be the size of a consumer statement, computed as its cardinality multiplied by the total tuple size. The cost components are defined as follows: I/O Cost (C_{io}): The total number of tuples accessed, regardless of filter or join conditions, because a full table scan reads all tuples from disk. CPU Cost (C_{cpu}): Computed as $size_{c_s}$ × system processing speed. Other Overhead (C_{other}): System-dependent costs (e.g., gathering statistics). The estimated cost of a full table scan is:

$$C_{s_c} = size_{c_s} \times C_{io} + C_{cpu} + C_{other} \tag{2}$$

Algorithm 2 Algorithm of the cost model for *what-if* analysis.

Input: Set of statements S, materialized temp tables, system parameters (I/O cost, CPU speed)
Output: Decision on whether to create an intermediate index

```
WhatIfAnalysis(S_c):
    FOREACH statement s in S:
        if isMaterialized(s) then
            CS ← getConumerStatement(s); //obtain Consumer statements
            t_s = getTemporaryTable(s)
            S_c = getConsumerStatements(t_s)
            IIndex(π_producer, θ_consumer) where θ_consumer ∈ π_producer //Create hypothetical index
            C_s = estimateFullTableScanCost(S_c) // Estimate cost without index (full table scan)
            I_ts = estimateIndexCreationCost(t_s) // Estimate cost of index creation
            C'_s = estimateIndexScanCost(S_c) // Estimate cost with index scan
            Benefit = sum(C_s) - (I_ts + sum(C'_s)) // Compute cost–benefit evaluation
            // Decide whether to create the index
            if Benefit > 0 then
                createIndex(t_s)
                updateExecutionPlan(s, t_s)
            end
        end
```

Function: estimateFullTableScanCost(S_c)

```
estimateFullTableScanCost(S_c):
    return sum(getSize(c) * getIOCost(c) + getCPUCost(c) + getSystemOverhead() for c in S_c)
```

Function: estimateIndexCreationCost(t_s)

```
estimateIndexCreationCost(t_s):
    return getSize(t_s) * getIOCost(t_s) + getCPUCost(t_s) + getSystemOverhead()
```

Function: estimateIndexScanCost(S_c)

```
estimateIndexScanCost(S_c):
    return sum(getSize(c) * (getIOCost(c) / getSelectivity(c)) + getCPUCost(c) + getSystemOverhead() for c in S_c)
```

3.2.2. Estimation of index creation cost on producer temporary table

Let $size_{I_{t_s}}$ be the index size, computed as the cardinality of the producer statement multiplied by the total size of indexed columns. The cost components are: I/O Cost (I_{io}): Equal to the producer statement's cardinality because index creation involves scanning, sorting, and filtering all rows. CPU Cost (I_{cpu}): Computed as $size_{I_{t_s}} \times$ system processing speed. Other Overhead (C_{other}): System-dependent costs. The estimated index creation cost is:

$$I_{t_s} = size_{I_{t_s}} \times I_{io} + I_{cpu} + C_{other} \tag{3}$$

3.2.3. Estimation of consumer statements cost with an index scan

To estimate the cost of a consumer statement with an index scan, we use the same equation as for a full table scan but adjust the I/O cost. The I/O cost for an index scan, C'_{io}, is determined by selectivity estimation [12]. Let $T_{consumer}$ be the cardinality of the consumer statement, C_{io} be the I/O cost without a filter or join condition. The estimated I/O cost with an index scan is: $C'_{io} = \frac{C_{io}}{T_{consumer}}$.

The equation uses selectivity estimation to reduce the I/O component of the index-scan cost. A lower selectivity results in fewer qualifying tuples, thereby decreasing the number of page accesses required during the index probe. This behavior is consistent with the optimizer's standard selectivity-based cost scaling. Furthermore, since database systems do not expose fine-grained internal timing for index creation on temporary tables, we empirically measured index-creation cost by comparing execution times with and without the index under identical conditions. This practical method preserves reproducibility and reflects actual system behavior.

We show the algorithm that performs a cost–benefit analysis using *what-if* analysis in Algorithm 2. $WhatIfAnalysis(S)$ evaluates whether creating an intermediate index is beneficial. It iterates through statements and checks if they produce a materialized temporary table. We apply the *AutoCox* method to create a hypothetical index on the producer temporary table. For each table, it retrieves its consumer statements and estimates the cost of full table scans (C_s), the cost of index creation (I_{t_s}), and the cost of index-based execution (C'_s). If using an index finds a benefit, it creates the index and updates the execution plan. The following functions are responsible to estimate C_s, I_{t_s}, and C'_s. $estimateFullTableScanCost(S_c)$ computes the total cost of scanning consumer statements without an index. It uses I/O cost, CPU cost, and system overhead. $estimateIndexCreationCost(t_s)$ estimates the cost of creating an index on a temporary table based on table size, I/O cost, CPU cost, and system overhead. $estimateIndexScanCost(S_c)$ computes the cost of scanning consumer statements with an index. It adjusts I/O cost using selectivity estimation.

Table 1
Nature of the S-SOAP dataset.

Name	Num. statement	Procedural pattern	Description
$Proc_1$	4	A statement with a consumer statement.	There is a producer statement and only a consumer statement that contains a filter condition. The producer statement produces a large amount of data for the consumer statement to process. A loop within a consumer statement repeatedly accesses the producer statement.
$Proc_2$	4	A statement with multiple consumer statements.	Multiple consumer statements consume a producer statement. We process all consumer statements in parallel, finally aggregating the results.
$Proc_3$	10	Multiple statements with multiple producer statement.	Each consumer statement consumes data from multiple producer statements with different conditions.
$Proc_4$	8	A series of dependent statements.	Only a consumer statement has a series of consumer statements from leaf statement to root. A consumer statement has only a producer statement and a consumer statement. The aggregated statement is in the loop.
$Proc_5$	13	A series of consumer statements inside a loop.	Only a consumer statement has a series of consumer statements from leaf statement to root. A consumer statement has only a producer statement and a consumer statement. The series of statements is in the loop.

3.3. Execution plan generation

To incorporate beneficial intermediate indexes into the physical execution plan, we insert index creation statements immediately after the corresponding producer statements. The selection of the indexed column is determined by our *what-if* analysis, which identifies the most relevant column from consumer statements to optimize data retrieval. For instance, in Fig. 3, the column d is selected for index creation on the temporary table T_6. To ensure uniqueness and prevent conflicts, we adopt a systematic naming convention for indexes. Given that an imperative program may generate multiple indexes, we name each index uniquely by appending a predefined string to the corresponding temporary table name. Since our approach ensures that at most one index is created per temporary table, this naming strategy eliminates potential conflicts. For example, the index created on the temporary table T_6 is named $index_T6$. In Fig. 3(c), we illustrate the incorporation of the index creation statement in the physical execution plan. Specifically, the following data definition language (DDL) statement is introduced immediately after statement 6:

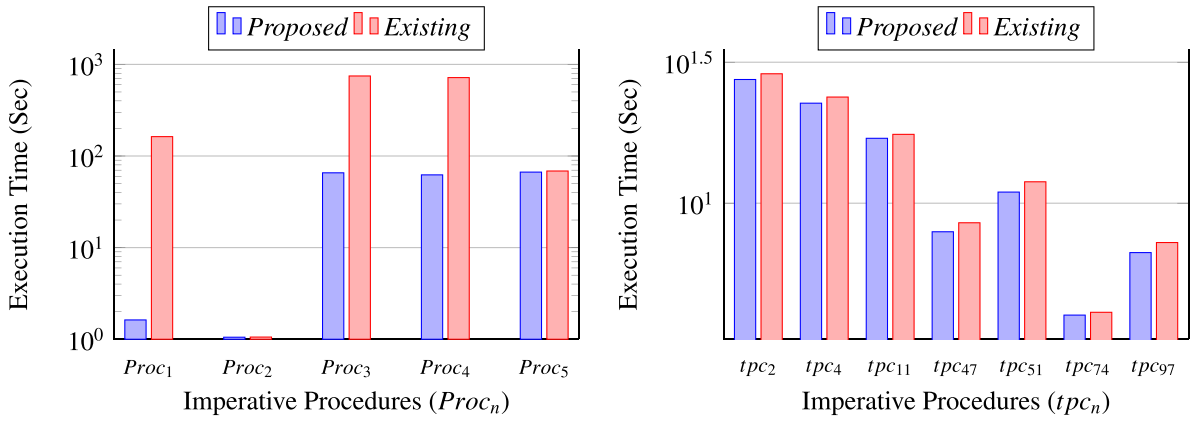
```
CREATE INDEX index_T6 ON T6 (d);
```

4. Experiment

We evaluated the effectiveness of our proposed strategy in optimizing materialized intermediate results utilizing intermediate indexes by comparing it to the state-of-the-art techniques and three distinct datasets.

4.1. Datasets

- **S-SOAP**: We use the S-SOAP (Structure of Snakes-on-a-Plan) dataset, which consists of 125 stored procedures organized into distinct procedural pattern classes. Each program has 25 variants, generated by systematically varying input parameters to progressively increase the number of consumer accesses to intermediate results. As not all datasets in the original Snakes-on-a-Plan benchmark [13] are suitable for materialization analysis, i.e., some programs lack reuse opportunities for intermediate results, we focus only on those with high potential for materialization. Our selected programs exhibit procedural structures that favor reuse and optimization of intermediate results. We design all datasets using the TPC-DS database schema as the underlying data source. Table 1 summarizes the procedural pattern classes and describes the intermediate indexes that represent potential materialization candidates. We further decompose each program into its corresponding statement graph, which captures the procedural control flow and data dependencies inherent in each pattern. For clarity, we denote five representative programs used in our evaluation as $Proc_1$, $Proc_2$, $Proc_3$, $Proc_4$, and $Proc_5$.
- **P-TPC-DS**: The procedural extension to SQL queries in the TPC-DS benchmark (P-TPC-DS) contains the augmentation of the imperative structure with the original TPC-DS queries. Since the original TPC-DS queries [14] represent the real-world workload, our evaluation with these augmented programs proves that we are able to apply our algorithm to any program. We augmented the TPC-DS queries by converting an SQL query to a statement in a stored procedure based on its statement graph. If a query contains a CTE-based query, we convert it into producer and consumer statements by storing the CTEs in temporary tables. We observe that out of 99 queries, there are 34 queries that utilize CTEs. Since there are no consumer statements in the remaining stand-alone queries, our algorithm does not find any potential intermediate index. Out of 34 procedures, we found



(a) Execution time evaluation on S-SOAP dataset.

(b) Execution time evaluation on P-TPC-DS dataset.

Fig. 6. Overall performance evaluation.

seven queries in which the producer statement generates a large data for the consumer statement. For these programs, the optimizer decides to materialize intermediate results for multiple consumer accesses. We include a control region (i.e., loop) in the aggregated statement in the seven programs to evaluate the multiple consumer accesses. We denote the seven programs as tpc_2, tpc_4, tpc_{11}, tpc_{47}, tpc_{51}, tpc_{74}, and tpc_{97}.

- **SQL-ProcBench**: We analyzed another well-known procedural extension to the SQL benchmarks, *SQL-ProcBench* [15]. Since the benchmark focuses on translating imperative logic into equivalent declarative SQLs, it prefers inlining in every example scenario. Thus, our algorithm does not find any potential intermediate index in this benchmark.

4.2. Experimental setup

We evaluate the effectiveness of our proposed intermediate index creation approach using a series of controlled experiments. The experiments are conducted on a machine equipped with an Intel Xeon 3.0 GHz CPU, 64 GB RAM, and a 2 TB SSD. We use PostgreSQL as the database system and utilize the PL/pgSQL dialect to create stored procedures. We used the state-of-the-art auto-indexing recommendation [10] method for UDF as our baseline. The baseline method creates an index on a base table with the awareness that batches of statements with temporary tables utilize the base table during execution.

4.3. Experimental results

4.3.1. Execution time analysis

To evaluate the efficiency of our *proposed* indexing approach, we conduct experiments using the S-SOAP and P-TPC-DS datasets. We compare the execution time of our method against the *existing* auto-indexing [10] technique. The evaluation considers two key performance metrics:

- *Execution Time* (T_q): The total time required to execute queries, including optimization time.
- Speedup (γ): The percentage improvement in query execution time, defined as: $\gamma = \frac{T_q^{\text{Auto-Indexing}} - T_q^{\text{Proposed}}}{T_q^{\text{Auto-Indexing}}} \times 100\%$

Fig. 6(a) presents the overall performance comparison, demonstrating the order-of-magnitude improvement achieved by our approach using the S-SOAP dataset. To account for loop regions in programs such as $Proc_1$, $Proc_3$, $Proc_4$, and $Proc_5$, we execute each program for 500 iterations. All experiments are conducted on a 1 GB database. Our proposed method achieves a 100% improvement over auto-indexing, making it 261 times faster on average. Specifically, for $Proc_1$, $Proc_2$, $Proc_3$, $Proc_4$, and $Proc_5$, our approach achieves 100%, 8%, 92%, 5%, and 3% speedup, respectively.

Case Study Analysis on the S-SOAP dataset: $Proc_1$ eliminates redundant full table scans. The auto-indexing method performs a full scan of the temporary table for each access from the consumer statement, leading to significant overhead. In contrast, our approach creates an index once and reuses it across multiple accesses, reducing execution time dramatically. $Proc_2$ reduces cost via single index usage. In this scenario, the primary optimization comes from reducing consumer statement costs by leveraging an intermediate index. $Proc_3$ leverages multiple indexes for complex queries. This case involves multiple producer statements and multiple filter and join conditions. Our method dynamically selects and applies multiple indexes, significantly reducing execution time compared to auto-indexing. Additionally, multiple consumer statements benefit from a single optimized index, further contributing to the speedup. Proc_4 optimizes multi-level producer–consumer relationships. This scenario consists of nested producer–consumer relationships,

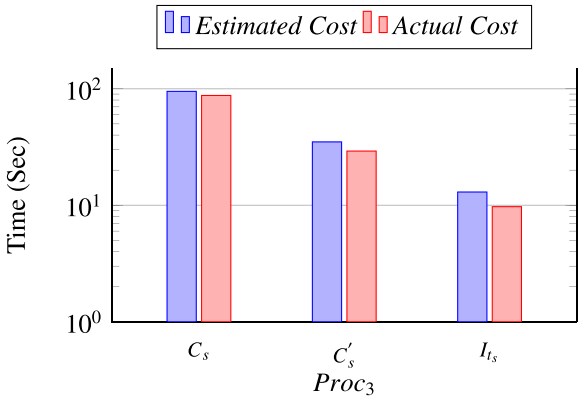


Fig. 7. Evaluation on cost model validation.

where each consumer statement benefits from an index at its respective level. Unlike auto-indexing, our approach selectively applies indexes without disrupting producer statements. $Proc_5$ shows limitations in the loop regions. Our method does not determine temporary tables within loop regions as candidates for indexing due to large index creation overheads. As a result, while some performance gains are observed, they are less significant compared to other cases.

Dataset-Specific Performance Differences: The S-SOAP dataset exhibits much larger performance gains than P-TPC-DS because the two workloads differ fundamentally in the structure and reuse patterns of intermediate results. S-SOAP programs are synthetically designed to include loop regions, repeated consumer accesses, and large intermediate result cardinalities, allowing the index creation cost to be amortized over many consumer statements. In contrast, P-TPC-DS contains far fewer reusable intermediate results; only 7 out of 99 transformed procedures generate materialized temporary tables with multiple consumers. As a result, opportunities for index reuse are limited, and the overall improvement is smaller. Therefore, the reported 261 × speed-up applies specifically to S-SOAP programs with high reuse intensity, while P-TPC-DS correctly shows moderate benefits due to fewer materialization opportunities.

4.3.2. Cost model validation

To validate our proposed cost model, we conduct experiments using the S-SOAP dataset. Since our cost model relies on estimated costs—such as the full table scan cost, index scan cost, and index creation cost (as formulated in Section 3.2), we assess these estimated costs against their actual execution-time measurements. The experiment shows the effectiveness of our cost model in accurately predicting index creation benefits. By leveraging estimated cardinality and selectivity metrics, our model successfully anticipates performance improvements, making it a reliable foundation for optimization decisions in creating an intermediate index.

Method: We empirically measure the actual execution costs using the following approach: We determine the execution cost of a consumer statement with a full table scan (C_s) by measuring the difference in execution time between two subsets of the program: (i) the execution time of the program up to the consumer statement and (ii) the execution time of the program excluding the consumer statement. Similarly, we measure the execution cost of a consumer statement using an intermediate index scan (C'_s) by computing the time difference between: (i) the execution time of the program up to the consumer statement with an index and (ii) the execution time of the program excluding the consumer statement. We determine the index creation cost (I_{t_s}) by measuring the difference between the full table scan cost and the index scan cost in the same execution environment.

Results and Analysis: We present our evaluation for $Proc_3$ in Fig. 7, as it contains multiple producer and consumer statements, making it an ideal candidate for validating our cost model. We compare estimated and actual costs for C_s, C'_s, and I_{t_s}. Our results indicate that the estimated costs closely align with the actual execution costs, confirming the accuracy of our cost model. Additionally, we check the benefit estimation using Eq. (1), showing that the real improvements in execution time are in line with what our cost model predicted. Moreover, we observe that we reduce the cost of consumer statements. For example, in $Proc_3$, we reduce the cost by a magnitude of 67%. The finding indicates that we successfully create an intermediate index on the producer's results that facilitates consumer queries.

4.3.3. Trends on multiple consumer accesses

This experiment evaluates the robustness of our proposed optimization method under varying consumer access patterns to the producer statement. We achieve this by modifying input parameters in programs that directly influence consumer statement access frequency.

Method: We conduct our evaluation using 100 programs from the S-SOAP dataset. Specifically, we show $Proc_1$, $Proc_3$, and $Proc_4$ since their loop regions introduce significant variability in consumer access behavior.

Results and Analysis: Fig. 8 illustrates the performance of our approach in handling multiple consumer accesses. The results indicate that as the number of consumer accesses increases, the performance improvement of our method grows exponentially. The existing trend shows how traditional methods find execution plans and train the ML models that increases execution time with

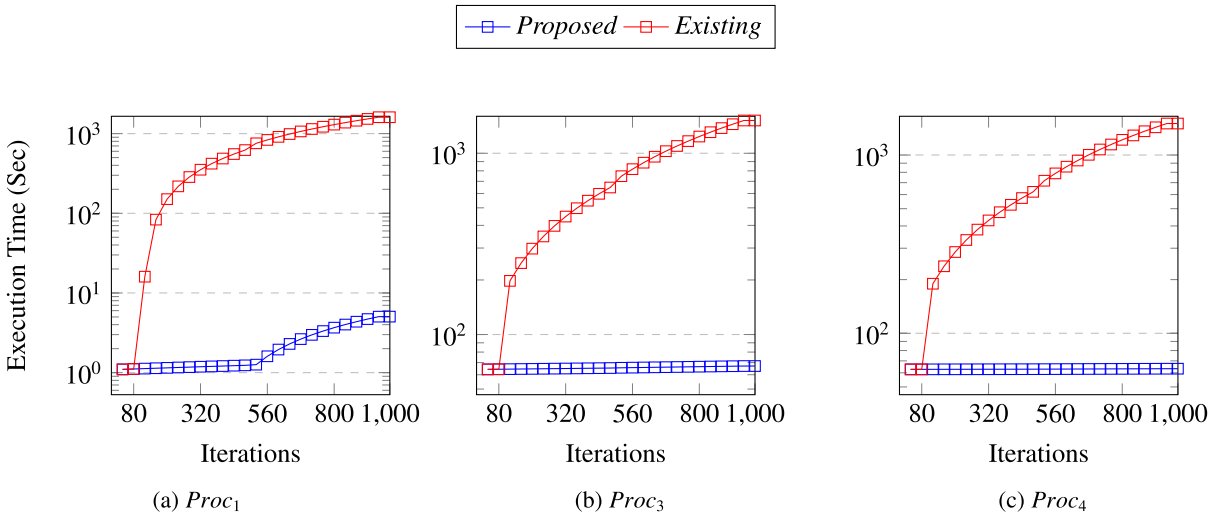


Fig. 8. Evaluation on trends of multiple consumer accesses.

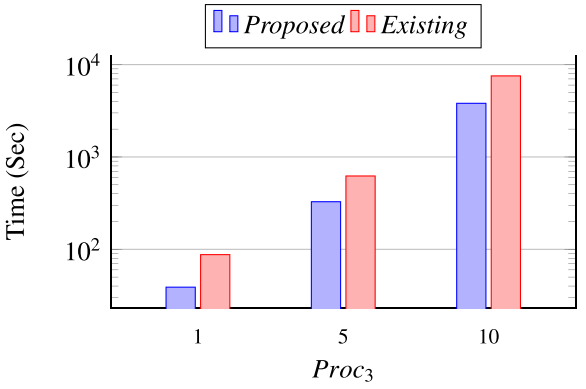


Fig. 9. Evaluation on intermediate index validity.

additional consumer access. *AutoCox* overcomes the limitation where it does not increase execution time with additional consumer accesses. It creates a better execution plan by reusing the same intermediate index across all accesses, avoiding redundant temporary table scans. Moreover, our cost model dynamically determines whether index creation is beneficial. The figure shows that no index is created when the number of consumer accesses falls below 82, 86, and 88 for $Proc_1$, $Proc_3$, and $Proc_4$, respectively. In these cases, our method defaults to using the existing physical execution plan rather than introducing an unnecessary index. Our method relieves developers, who rely on intuition and experience. By automating index selection, our approach reduces the reliance on manual tuning. Hardcoded index creation degrades performance when consumer accesses fall below the identified thresholds because the combined cost of index creation and indexed execution exceeds that of a full table scan. Our method avoids this inefficiency by adapting to workload characteristics in real-time.

4.3.4. Intermediate index validity evaluation

This experiment evaluates the effectiveness and correctness of our algorithm for managing intermediate indexes within the scope of a single program execution. Our approach ensures that intermediate indexes are created during execution and automatically dropped upon program completion, thereby guaranteeing access to up-to-date data and preventing static index reuse. To assess this behavior, we executed the same program repeatedly at high frequency, verifying two main properties: (i) the correctness of the program rewriting technique, ensuring that each invocation generates a correct and isolated execution plan with no interference between successive runs. (ii) Index lifecycle isolation, confirming that index creation in one invocation does not conflict with or persist into another. As shown in Fig. 9, we use a computationally intensive program, $Proc_3$, to analyze the performance across increasing execution frequencies. The results demonstrate that our algorithm consistently outperforms the baseline approach by maintaining performance stability and avoiding index conflicts or stale data issues.

5. Related works

Many research studies have been conducted on the optimization of imperative programs [6,16–20]. Because of their control flow, intermediate result processing, and dependency management, these programs, which combine SQL searches with procedural logic, offer flexibility and reusability but also present special optimization problems [21–23]. For years, researchers explored various strategies to optimize imperative programs, such as inlining and materialization, and advanced techniques like automatic indexing [24–27] and dynamic indexing [28–36]. However, the problem of optimizing intermediate results, especially when no index exists, remains largely underexplored.

5.1. The no-index problem

The current optimization techniques are significantly restricted by their dependence on preexisting indexes. The optimizer conducts a full table scan on large datasets when there is no index on a transient table, which is costly. This problem severely affects imperative programs, where temporary tables are frequently used to store intermediate results. For instance, Das et al. [10] introduced a workload-based methodology to optimize imperative programs by establishing indexes on source tables (i.e., tables that are stored in the database). Nevertheless, their approach does not consider the optimization of indexes on ephemeral tables.

5.2. Automatic indexing: A step forward

The limitations of manual index creation have been addressed by researchers who have investigated automatic indexing techniques. Hypothetical indexes or *what-if* analysis are a promising approach to assess the potential advantages of indexes without the necessity of their creation. This method enables optimizers to simulate the effect of indexes on query performance, thereby facilitating the making of more informed decisions. These methods substantially enhance query performance by analyzing query workloads and recommending indexes for source tables. For example, Chaudhuri and Narasayya [7] developed the AutoAdmin "what-if" index analysis utility, which assesses the potential advantages of indexes on source tables. In the same vein, Gao et al. [25] introduced a learned cost estimator for the purpose of automatic index selection. Nevertheless, these methods are limited to the database level and do not apply to transient tables within imperative programs. Although they are a substantial improvement, they neglect to consider the dynamic character of intermediate results in procedural logic. Conversely, as we concentrate on the temporary index creation within the imperative program, we are obligated to conduct a *what-if* analysis and estimate the overhead cost of index creation in order to assess their prospective benefits.

5.3. Dynamic indexing: Experimental and specialized systems

In the past few years, researchers have begun investigating dynamic indexing techniques, which involve the creation of indexes on-the-fly for specific workflows or queries. In specialized systems, such as real-time analytics platforms or in-memory databases, these methods are particularly beneficial. For example, Tiresias, a system that facilitates predictive autonomous storage and indexing, was proposed by Abebe et al. [26]. Dynamic indexing has demonstrated potential; however, it has not been extensively implemented in conventional relational databases, particularly for imperative programs. In addition, the current methodologies concentrate on persistent data and fail to address the distinctive obstacles associated with transient tables.

5.4. Workarounds when no index exists

Developers frequently implement remedies, including manually generating indexes or rewriting queries, to mitigate the effects of full table scans when indexes are not present on temporary tables. For example, Ramachandra et al. [19] proposed techniques for optimizing imperative programs by converting procedural logic into set-oriented operations, thereby reducing the necessity for transient tables. In the same vein, Gupta et al. [18] implemented custom aggregates to supplant cursor loops, thereby enhancing performance without having to rely on indexes. Although some of these remedies are effective, they do not offer a systematic solution for optimizing access to intermediate results.

5.5. Relation to learned query optimizers

Recent learned optimizers (e.g., Neo [1], Bao [2], Balsa [3], Lero [4]) focus on predicting cardinalities, join orders, or plan choices, but they assume that the set of available indexes is fixed. Consequently, they do not reason about intermediate results produced inside imperative programs. *AutoCox* complements these methods by enriching the optimizer's candidate plan space with intermediate indexes that would otherwise be absent. Learned optimizers can therefore benefit from *AutoCox* by training on richer execution plans that include improved access paths for temporary results.

5.6. The intermediate index: Bridging the gap

In contrast, the narrative of imperative program optimization is one of continuous evolution, from the development of early techniques such as inlining and materialization to the development of advanced methods such as automatic and dynamic indexing. Nevertheless, optimizing intermediate results (particularly in the absence of an index) continues to be a significant obstacle. Under these circumstances, we identified a deficiency in the existing literature that pertains to cost-based optimization in order to suggest indexes on transient tables for the physical execution plan of imperative programs in relational databases.

6. Conclusion

In this work, we introduced a consumer-aware auto-indexing approach that optimizes procedural SQL query processing by strategically identifying intermediate indexes for model training in a learned query optimizer. Unlike traditional auto-indexing techniques, which primarily focus on static queries, our method dynamically adapts to execution patterns by analyzing consumer access behaviors. The introduction of a *what-if* analysis in our cost model guarantees that indexes are generated only when they are beneficial. The effectiveness of our method is demonstrated by our experimental evaluation. We achieve huge performance improvements in comparison to previously employed auto-indexing methods. In situations where there are multiple consumer accesses, our approach performs well as it prevents redundant full-table queries and facilitates efficient data retrieval. The utilization of consumer-aware indexing, as evidenced by the performance trends, enhances the training of ML-based query optimizers. They are able to develop more effective execution plans for procedural SQL queries.

Our future work will include enhancing our approach to evolving workloads within a single database system. It includes dynamically adjusting intermediate indexes based on workload shifts. We believe that our proposed method helps database researchers to improve database performance in increasingly complex applications.

CRediT authorship contribution statement

Md Arif Rahman: Writing – review & editing, Writing – original draft, Visualization, Software, Methodology, Formal analysis, Data curation, Conceptualization. **Syed Jalaluddin Hashmi:** Writing – review & editing. **Kethsiya Gnanajothy:** Writing – review & editing. **Young-Koo Lee:** Writing – review & editing, Validation, Supervision, Resources, Project administration, Investigation, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government(MSIT) (No. RS-2022-00155911, Artificial Intelligence Convergence Innovation Human Resources Development (Kyung Hee University)).

Data availability

Data will be made available on request.

References

[1] R. Marcus, P. Negi, H. Mao, C. Zhang, M. Alizadeh, T. Kraska, O. Papaemmanouil, N. Tatbul, Neo: A learned query optimizer, 2019, arXiv preprint arXiv:1904.03711.
[2] R. Marcus, P. Negi, H. Mao, N. Tatbul, M. Alizadeh, T. Kraska, Bao: Making learned query optimization practical, in: Proceedings of the 2021 International Conference on Management of Data, 2021, pp. 1275–1288.
[3] Z. Yang, W.-L. Chiang, S. Luan, G. Mittal, M. Luo, I. Stoica, Balsa: Learning a query optimizer without expert demonstrations, in: Proceedings of the 2022 International Conference on Management of Data, 2022, pp. 931–944.
[4] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, J. Zhou, Lero: A learning-to-rank query optimizer, 2023, arXiv preprint arXiv:2302.06873.
[5] P. Sulimov, C. Lehmann, K. Stockinger, GenJoin: Conditional generative Plan-to-Plan query optimizer that learns from subplan hints, 2024, arXiv preprint arXiv:2411.04525.
[6] K. Park, H. Seo, M.K. Rasel, Y.-K. Lee, C. Jeong, S.Y. Lee, C. Lee, D.-H. Lee, Iterative query processing based on unified optimization techniques, in: Proceedings of the 2019 International Conference on Management of Data, 2019, pp. 54–68.
[7] S. Chaudhuri, V. Narasayya, AutoAdmin "what-if" index analysis utility, ACM SIGMOD Rec. 27 (2) (1998) 367–378.
[8] T.P.G.D. Group, Introducing HypoPG, hypothetical indexes for PostgreSQL, 2015, https://www.postgresql.org/about/news/introducing-hypopg-hypothetical-indexes-for-postgresql-1593/. (Online Accessed June 2015).
[9] Oracle-Base, Automatic indexing (DBMS_AUTO_INDEX) in oracle database 19c, 2021, https://oracle-base.com/articles/19c/automatic-indexing-19c. (Online Accessed April 2021).

[10] S. Das, M. Grbic, I. Ilic, I. Jovandic, A. Jovanovic, V.R. Narasayya, M. Radulovic, M. Stikic, G. Xu, S. Chaudhuri, Automatically indexing millions of databases in microsoft azure sql database, in: Proceedings of the 2019 International Conference on Management of Data, 2019, pp. 666–679.

[11] S. Muchnick, et al., Advanced Compiler Design Implementation, Morgan kaufmann, 1997.

[12] H. Garcia-Molina, Database Systems: The Complete Book, Pearson Education India, 2008.

[13] T. Fischer, D. Hirn, T. Grust, Snakes on a plan: Compiling python functions into plain SQL queries, in: Proceedings of the 2022 International Conference on Management of Data, 2022, pp. 2389–2392.

[14] T.P.P. Council, TPC BENCHMARK^{TM} DS version 2.13.0, 2020, http://www.tpc.org/tpcds/. (Online Accessed August 2020).

[15] S. Gupta, K. Ramachandra, Procedural extensions of SQL: Understanding their usage in the wild, Proc. VLDB Endow. 14 (8) (2021) 1378–1391.

[16] M. Chavan, R. Guravannavar, K. Ramachandra, S. Sudarshan, DBridge: A program rewrite tool for set-oriented query execution, in: 2011 IEEE 27th International Conference on Data Engineering, IEEE, 2011, pp. 1284–1287.

[17] K.V. Emani, K. Ramachandra, S. Bhattacharya, S. Sudarshan, Extracting equivalent sql from imperative code in database applications, in: Proceedings of the 2016 International Conference on Management of Data, 2016, pp. 1781–1796.

[18] S. Gupta, S. Purandare, K. Ramachandra, Aggify: Lifting the curse of cursor loops using custom aggregates, in: Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data, 2020, pp. 559–573.

[19] K. Ramachandra, K. Park, K.V. Emani, A. Halverson, C. Galindo-Legaria, C. Cunningham, Froid: Optimization of imperative programs in a relational database, Proc. VLDB Endow. 11 (4) (2017) 432–444.

[20] V. Simhadri, K. Ramachandra, A. Chaitanya, R. Guravannavar, S. Sudarshan, Decorrelation of user defined function invocations in queries, in: 2014 IEEE 30th International Conference on Data Engineering, IEEE, 2014, pp. 532–543.

[21] M.A. Rahman, Y.-K. Lee, Forester: Approximate processing of an imperative procedure for Query-Time exploratory data analysis in a relational database, Electronics 13 (4) (2024) 759.

[22] M.A. Rahman, Y.-K. Lee, Iterative skyline query processing using Rule-based optimization techniques, in: Korean DataBase Conference 2025, Vol. 27, 2025.

[23] M.A. Rahman, Y.-K. Lee, AutoCache: Efficient execution of UDF through the detection of cached variables for the analytical analysis on federated databases, in: The 10th International Conference on Big Data Applications and Services, Vol. 10, (ISSN 2466-135X) Korea Big Data Service Society, 2022, p. 143, https://www.jbig.or.kr.

[24] D. Das, J. Yan, M. Zait, S.R. Valluri, N. Vyas, R. Krishnamachari, P. Gaharwar, J. Kamp, N. Mukherjee, Query optimization in Oracle 12c database in-memory, Proc. VLDB Endow. 8 (12) (2015) 1770–1781.

[25] J. Gao, N. Zhao, N. Wang, S. Hao, H. Wu, Automatic index selection with learned cost estimator, Inform. Sci. 612 (2022) 706–723.

[26] M. Abebe, H. Lazu, K. Daudjee, Tiresias: enabling predictive autonomous storage and indexing, Proc. VLDB Endow. 15 (11) (2022) 3126–3136.

[27] J. Kossmann, S. Halfpap, M. Jankrift, R. Schlosser, Magic mirror in my hand, which is the best in the land? an experimental evaluation of index selection algorithms, Proc. VLDB Endow. 13 (12) (2020) 2382–2395.

[28] M. Hünemörder, P. Kröger, M. Renz, Towards a learned index structure for approximate nearest neighbor search query processing, in: N. Reyes, R. Connor, N.M. Kriege, D. Kazempour, I. Bartolini, E. Schubert, J. Chen (Eds.), Similarity Search and Applications - 14th International Conference, SISAP 2021, Dortmund, Germany, September 29 - October 1, 2021, Proceedings, in: Lecture Notes in Computer Science, Vol. 13058, Springer, 2021, pp. 95–103, http://dx.doi.org/10.1007/978-3-030-89657-7_8.

[29] S. Herath, M. Roughan, G. Glonek, Em-K indexing for approximate query matching in Large-scale ER, 2021, CoRR abs/2111.04070 arXiv:2111.04070 URL https://arxiv.org/abs/2111.04070.

[30] J. Jo, J. Seo, J. Fekete, PANENE: A progressive algorithm for indexing and querying approximate k-Nearest neighbors, IEEE Trans. Vis. Comput. Graph. 26 (2) (2020) 1347–1360, http://dx.doi.org/10.1109/TVCG.2018.2869149.

[31] D. Chen, G. Sun, N.Z. Gong, Efficient approximate top-*k* query algorithm using cube index, in: X. Du, W. Fan, J. Wang, Z. Peng, M.A. Sharaf (Eds.), Web Technologies and Applications - 13th Asia-Pacific Web Conference, APWeb 2011, Beijing, China, April 18-20, 2011. Proceedings, in: Lecture Notes in Computer Science, Vol. 6612, Springer, 2011, pp. 155–167, http://dx.doi.org/10.1007/978-3-642-20291-9_17.

[32] Y. Shin, J. Ahn, D. Im, Join optimization for inverted index technique on relational database management systems, Expert Syst. Appl. 198 (2022) 116956, http://dx.doi.org/10.1016/j.eswa.2022.116956.

[33] M. Kvet, Relational data index consolidation, in: 28th Conference of Open Innovations Association, FRUCT 2021, Moscow, Russia, January 27-29, 2021, IEEE, 2021, pp. 215–221, http://dx.doi.org/10.23919/FRUCT50888.2021.9347614.

[34] M. Yahyaoui, S. Amjad, L. Benameur, I. Jellouli, Efficient of bitmap join indexes for optimising star join queries in relational data warehouses, Int. J. Comput. Intell. Stud. 9 (3) (2020) 220–233, http://dx.doi.org/10.1504/IJCISTUDIES.2020.109604.

[35] M. Korytkowski, Novel visual information indexing in relational databases, Integr. Comput. Aided Eng. 24 (2) (2017) 119–128, http://dx.doi.org/10.3233/ICA-160534.

[36] J.M. Medina, C.D. Barranco, O. Pons, Indexes for necessity queries. Implementation and performance evaluation on a fuzzy Object-Relational database management system, in: 2018 IEEE International Conference on Fuzzy Systems, FUZZ-IEEE 2018, Rio de Janeiro, Brazil, July 8-13, 2018, IEEE, 2018, pp. 1–6, http://dx.doi.org/10.1109/FUZZ-IEEE.2018.8491608.



Md Arif Rahman is a Ph.D. candidate in Computer Science and Engineering at Kyung Hee University, South Korea. He earned his B.S. and M.S. in Computer Science from IUT, Bangladesh, in 2009 and 2011. Since 2012, he has served as a faculty member at Jashore University of Science and Technology, becoming an assistant professor in 2015. Currently, he conducts research at the Data and Knowledge Engineering Lab under Professor Young-Koo Lee. His research interests include query processing, learned query optimization, and vector databases. He actively contributes to database research through publications, conferences, and academic collaboration.



Syed Jalaluddin Hashmi is currently pursuing a Ph.D. in Artificial Intelligence at Kyung Hee University, South Korea. He holds a Master's degree in Computer Science from FAST National University, Islamabad, Pakistan, where he also completed his undergraduate studies with a strong academic record. His research interests include machine learning (ML), deep learning (DL), large language models (LLMs), query optimization, query processing, graph data management, and database systems (DBS). His ongoing work focuses on integrating advanced ML techniques into data-intensive systems for performance-aware optimization.



Kethsiya Gnanajothy is a Ph.D. candidate in the Department of Computer Science and Engineering at Kyung Hee University, South Korea. She earned her Bachelor's degree in Computer Science and Engineering from Anna University, India. Currently, Kethsiya serves as a researcher at the Data and Knowledge Engineering Laboratory (DKE Lab) at Kyung Hee University. Her primary research interests include query optimization, machine learning, and vector databases.



Young-Koo Lee (Member,IEEE) received his B.S., M.S., and Ph.D. in computer science from KAIST, South Korea, in 1992, 1994, and 2002, respectively. He was a Postdoctoral Researcher at the University of Illinois at Urbana–Champaign and KAIST's AITrc until 2004. He then joined Kyung Hee University, Global Campus, where he became a Professor in the College of Software in 2015. Prof. Lee has published extensively in top-tier venues and has contributed significantly to data science and GNN-based learning. His research interests include graph neural networks, data mining, OLAP, query optimization, and big data. He is recognized for his work in large-scale graph processing.